SOC LEVEL 1 • MALWARE TRIAGE

Shadow Trace

TryHackMe Lab Investigation Report

Case type	Suspicious Windows executable / possible downloader
Sample	windows-update.exe
Environment	TryHackMe Shadow Trace lab; static analysis and alert correlation
Primary tools shown	PE-bear, PowerShell Get-FileHash, CyberChef
Evidence set	8 user-uploaded screenshots, individually numbered E-01 to E-08
Report date	22 October 2026

Executive assessment: the evidence is consistent with a suspicious 64-bit Windows executable masquerading as an updater. Static strings expose download-related URLs/domains, and an imported socket library is consistent with network capability. Public walkthroughs also describe alerts involving obfuscated PowerShell and JavaScript download logic. Treat the sample as malicious in the lab; do not execute it on a production system.

Evidence numbering note: every screenshot in the appendix is numbered E-01 onward, and findings below cite those same IDs. This avoids mismatching captions and screenshots.

1. Executive Summary

The Shadow Trace room asks the analyst to inspect a suspicious file found at C:\Users\DFIRUser\Desktop\windows-update.exe, extract indicators of compromise (IOCs), and correlate EDR-style alerts to understand the likely activity. TryHackMe describes the objectives as IOC extraction, alert correlation, and basic SOC triage. [1]

The supplied screenshots show PE-bear identifying the binary as AMD64/64-bit, PowerShell calculating its SHA-256 hash, embedded strings containing a URL associated with an apparent security-update executable, the WS2_32.dll networking library, and CyberChef decoding decimal character codes into a second suspicious URL. Public walkthroughs further document a Base64-obfuscated PowerShell URL and an alert where Chrome-related JavaScript downloads a file named test.txt. [2][3]

Scope limitation: the supplied screenshots do not include all alert-dashboard screenshots or endpoint telemetry. The PowerShell/Chrome behavior below is sourced from public walkthroughs and should be treated as lab-context evidence, not as independently visible in the uploaded screenshots.

2. Scope and Method

• Static file triage: inspect PE headers, compute a cryptographic hash, extract strings, review imports.
• IOC extraction: record file hash, URL/domain indicators, and relevant library names.
• Alert correlation: compare the observed sample IOCs with the PowerShell and Chrome alert behaviors documented in walkthroughs.
• Response planning: prioritize isolation, evidence preservation, IOC blocking, endpoint hunting, and prevention controls.

3. Evidence Register

ID	Uploaded screenshot	What it supports
E-01	Screenshot 2026-09-25 182222(1).png	PE-bear shows Machine value 8664 / AMD64 (x64), consistent with a 64-bit Windows PE file.
E-02	Screenshot 2026-09-25 182535(1).png	Get-FileHash output records the SHA-256 hash for windows-update.exe.
E-03	Screenshot 2026-09-25 182643(1).png	PE-bear Strings view shows a hard-coded URL beginning http://tryhatme.com/update/security-update...
E-04	Screenshot 2026-09-25 182804(1).png	Strings view shows an encoded/path-bearing tryhatme.com string and other strings; walkthroughs identify responses.tryhatme.com as an IOC.
E-05	Screenshot 2026-09-25 182811(1).png	A wider view of extracted strings shows the updater-like URL embedded in the executable.
E-06	Screenshot 2026-09-25 183310(1).png	PE-bear Imports tab lists WS2_32.dll, a Windows socket/networking library.
E-07	Screenshot 2026-09-25 183816(1).png	CyberChef converts decimal character codes to https://reallysecureupdate.tryhatme.com/update.exe.
E-08	Screenshot 2026-09-25 183923(1).png	The screenshot records completion of the Shadow Trace room.

4. Findings — Individual PoC / Validation for Each

These are separate findings, not a combined PoC table. Each item includes evidence references, interpretation, validation steps, and mitigation.

Finding 01: Masquerading executable name

Severity: High — suspicious naming

Evidence: The file is named windows-update.exe; see E-01 and E-02.

Analysis: The filename imitates a legitimate operating-system update component. Naming alone is not proof of malware, but it warrants validation of signature, origin, path, and behavior.

Proof of concept / validation:

```
File: C:\Users\DFIRUser\Desktop\windows-update.exe
Check digital signature and publisher, file path, creation/download source, and whether the binary is approved by IT.
```

Mitigation: Do not trust filenames alone. Use application allowlisting, trusted software distribution, code-signing validation, and EDR detections for executables in user-writable paths.

Finding 02: 64-bit PE binary identified

Severity: Informational — triage attribute

Evidence: E-01 shows PE-bear File Header: Machine 8664 / AMD64 (K8).

Analysis: The PE header identifies an AMD64/x64 Windows executable. Architecture helps select appropriate analysis tools but is not itself a vulnerability.

Proof of concept / validation:

```
In PE-bear, inspect File Header → Machine. The screenshot shows 8664, mapped to AMD64 / 64-bit.
```

Mitigation: Record architecture as a triage attribute and continue with signature, hash, strings, imports, and behavior analysis.

Finding 03: File hash collected for correlation

Severity: High value IOC

Evidence: E-02 shows PowerShell Get-FileHash -Algorithm SHA256 output.

Analysis: A SHA-256 hash is a stable content identifier for this exact sample and can support searching across EDR, antivirus, email, and threat-intelligence systems.

Proof of concept / validation:

```
PowerShell command shown in the screenshot:
Get-FileHash -Algorithm SHA256 .\windows-update.exe
Observed hash: B2A88DE3E3BCFAE4A4B38FA36E884C586B5CB2C2C283E71FBA59EFDB9EA64BFC
```

Mitigation: Search the exact hash across endpoints and mail gateways, quarantine/block it where supported, and preserve the sample in controlled evidence storage. Hashes can change if the file changes.

Finding 04: Embedded download URL in binary

Severity: High — network IOC

Evidence: E-03 and E-05 show a URL string in PE-bear. Public walkthroughs report http://tryhatme.com/update/security-update.exe. [2][3]

Analysis: A hard-coded URL ending in security-update.exe is inconsistent with a file that has not been verified as a legitimate updater. It may be used to retrieve a second-stage payload.

Proof of concept / validation:

```
Use PE-bear Strings or Sysinternals strings to extract URLs without executing the file. Compare exact strings against the screenshot and record the IOC defanged in reports as hxxp://tryhatme[.]com/update/security-update[.]exe.
```

Mitigation: Block the exact URL/domain through DNS, proxy, secure web gateway, and EDR controls after validation; hunt proxy/DNS logs for previous access. Do not visit the URL from a normal workstation.

Finding 05: Suspicious related domain

Severity: High — network IOC

Evidence: E-04 shows suspicious tryhatme.com-related string content. Public walkthroughs identify responses.tryhatme.com as a domain IOC. [2][3]

Analysis: A related domain string can indicate infrastructure used for callbacks, responses, or payload delivery. A string in a binary proves presence in the file, not that a connection succeeded.

Proof of concept / validation:

```
Extract all strings containing 'tryhatme' or '.com'; compare the exact hostnames with the public walkthrough and the sample’s network telemetry. IOC to record: responses[.]tryhatme[.]com.
```

Mitigation: Search DNS, proxy, firewall, and EDR telemetry for the domain. Block only after validation and retain logs for scope assessment.

Finding 06: Networking library import

Severity: Medium/High — capability indicator

Evidence: E-06 shows WS2_32.dll in PE-bear Imports.

Analysis: WS2_32.dll is the Windows Sockets library used by applications that perform network communication. Its presence supports network capability but does not, by itself, prove malicious network activity.

Proof of concept / validation:

```
Inspect the PE import table in PE-bear and record WS2_32.dll. Correlate with observed outbound DNS/HTTP(S) connections and process telemetry.
```

Mitigation: Use EDR/network monitoring to identify destination domains, process ancestry, timestamps, and connection outcomes. Interpret imports alongside behavior, not in isolation.

Finding 07: Obfuscated URL in Chrome alert

Severity: High — download behavior documented in walkthrough

Evidence: E-07 shows CyberChef converting decimal character codes to https://reallysecureupdate.tryhatme.com/update.exe. Public walkthroughs report the associated Chrome alert and filename test.txt. [3][4]

Analysis: JavaScript can encode a URL as decimal character codes to make static review harder. The decoded URL and scripted file download behavior are consistent with a browser-based payload retrieval step.

Proof of concept / validation:

```
In an offline copy of the alert, decode the decimal array with CyberChef From Decimal (comma
delimiter). Expected URL: hxxps://reallysecureupdate[.]tryhatme[.]com/update.exe.
Walkthrough-reported saved filename: test.txt.
```

Mitigation: Block the URL/domain, inspect browser download history and file-creation telemetry, and search for test.txt by path, hash, creation time, and process lineage. Preserve the downloaded file for sandbox analysis.

Finding 08: PowerShell download-and-execute behavior

Severity: Critical if confirmed on host

Evidence: Not shown directly in the supplied screenshots; public walkthroughs document a PowerShell alert that Base64-decodes a URL and pipes downloaded content to IEX. [2][4]

Analysis: The walkthrough's command decodes to https://tryhatme.com/dev/main.exe and uses Invoke-Expression (IEX) on downloaded content. This is a high-risk pattern because it can execute remote script content in memory. It is lab-derived, not direct proof from the user's screenshot set.

Proof of concept / validation:

```
For authorized alert review, decode the Base64 argument from the captured alert without running it.
Walkthrough-reported URL: hxxps://tryhatme[.]com/dev/main.exe. Review PowerShell logs, command-line
telemetry, parent process, and outbound connections.
```

Mitigation: Isolate the endpoint if execution is suspected; collect PowerShell Operational logs, EDR process tree, memory/volatile evidence per policy, DNS/proxy logs, and persistence artifacts. Restrict PowerShell where feasible, enable script-block logging and AMSI/EDR protections, and block the IOC.

Finding 09: Potential second-stage download chain

Severity: High — inferred chain

Evidence: E-03/E-05 show the embedded security-update URL; E-07 shows the decoded Chrome alert URL. Public walkthroughs document both the Chrome download and PowerShell URL. [2][3][4]

Analysis: Taken together, the lab appears to model a staged chain: suspicious updater-like executable contains network indicators; alert evidence shows obfuscated download URLs; a browser-related script downloads a file under an unrelated name. Exact process order and successful execution must be established from alert timestamps and endpoint logs.

Proof of concept / validation:

```
Build a timeline from alert timestamps, process creation events, DNS/proxy events, file creation, and
file hashes. Validate parent-child process relationships instead of relying on names alone.
```

Mitigation: Contain the endpoint, identify all files downloaded in the same time window, hunt the IOCs fleet-wide, and verify no persistence or lateral movement occurred before closing the incident.

5. IOC Register

Type	Value (defanged where applicable)	Source / confidence
Filename	windows-update.exe	E-01/E-02; observed
SHA-256	b2a88de3e3bcfae4a4b38fa36e884c586b5cb2c2c283e71fba59efdb9ea64bfc	E-02; observed
URL	hxxp://tryhatme[.]com/update/security-update[.]exe	E-03/E-05 and walkthrough; observed string
Domain	responses[.]tryhatme[.]com	Public walkthrough; cross-check exact string in original sample
URL	hxxps://tryhatme[.]com/dev/main.exe	Walkthrough alert analysis; not visible in uploaded screenshots
URL	hxxps://reallysecureupdate[.]tryhatme[.]com/update.exe	E-07; decoded from decimal string
Filename	test.txt	Public walkthrough of Chrome alert; validate against alert screenshot/logs
Library	WS2_32.dll	E-06; observed import

6. Likely Attack Reconstruction

1. A suspicious executable named windows-update.exe is present on the user's desktop. Its PE header confirms a 64-bit Windows binary, but the filename alone does not establish legitimacy (E-01).
2. Static inspection yields a SHA-256 fingerprint and embedded URL/domain strings, including a URL suggesting retrieval of security-update.exe (E-02 to E-05).
3. The imported WS2_32.dll indicates that the program can use Windows socket APIs; it is a capability indicator, not proof of a completed connection (E-06).
4. In the room's alert-analysis task, public walkthroughs describe a PowerShell command that decodes a Base64 URL and pipes downloaded content to IEX, and a Chrome-related JavaScript alert that decodes a URL and downloads a file as test.txt. The Chrome URL is confirmed by the user's CyberChef screenshot (E-07).
5. The likely objective is retrieval and execution of additional payload content. Confirming whether execution succeeded, whether persistence was created, and whether data was exfiltrated requires the alert timeline and endpoint telemetry.

7. Mitigation and SOC Response

Immediate triage

• Isolate the affected host from the network if execution or active connections are suspected, following incident-response policy.
• Preserve the original executable, SHA-256, file path, timestamps, PE metadata, EDR alerts, process tree, PowerShell logs, browser download history, DNS/proxy/firewall records, and relevant Windows event logs.
• Search the fleet for the hash, filename, domains, URLs, test.txt, and related process command lines. Block validated IOCs in EDR, DNS, proxy, and mail/web controls.
• Analyze the sample only in an isolated malware-analysis environment. Do not execute it on a normal endpoint or browse the malicious URLs directly.

Preventive controls

• Enforce application control/allowlisting and restrict execution from user-writable folders such as Desktop and Downloads.
• Enable EDR behavioral detections for suspicious PowerShell download-and-execute patterns, encoded commands, browser-originated script downloads, and unusual outbound connections.
• Enable PowerShell Script Block Logging and appropriate module logging; centralize Windows event and EDR telemetry in the SIEM.
• Apply web/DNS filtering and block confirmed malicious domains and URLs; monitor newly observed domains and unusual executable downloads.
• Use least privilege, patch operating systems and applications, and maintain tested incident-response playbooks for malware alerts.
• Tune detections with known-good software baselines to reduce false positives.

Example detection logic (conceptual)

Raise a high-priority correlation when an executable in a user-writable directory is followed by PowerShell with encoded/decoded download logic or IEX, and a network connection to one of the extracted IOCs. Correlate by host, user, process lineage, and a short time window; do not alert on WS2_32.dll import alone.

8. Final Conclusion

The evidence supports classifying windows-update.exe as suspicious and likely malicious in the Shadow Trace lab. The binary is 64-bit, has a recorded SHA-256 hash, contains updater-themed download URL strings, and imports WS2_32.dll. The user's CyberChef screenshot decodes a second URL associated with the Chrome alert. Public walkthroughs describe a broader chain involving obfuscated PowerShell download-and-execute behavior and a browser-triggered download saved as test.txt. [2][3][4]

Most likely attack method: masquerading as a Windows updater, using embedded or obfuscated URLs to retrieve additional content, and leveraging PowerShell/JavaScript download behavior to stage a payload. Mitigation should focus on containment, IOC-based hunting, endpoint timeline reconstruction, blocking confirmed infrastructure, and stronger application/PowerShell/EDR controls. The supplied screenshots do not prove the sample executed successfully, established persistence, or exfiltrated data.

9. References and Walkthroughs

[1] TryHackMe, Shadow Trace room overview and learning objectives: https://tryhackme.com/room/shadowtrace

[2] AudiTTRSi, "TryHackMe - Shadow Trace Writeup" (5 Feb 2026): https://audittrsi.blogspot.com/2026/02/tryhackme-shadow-trace-writeup.html

[3] HackMD, "Shadow Trace — TryHackMe" (walkthrough with static-analysis and alert-analysis steps): https://hackmd.io/K2N_mUPDSraZKb4nuryQ3w

[4] Osman Dhaqane, "Shadow Trace TryHackMe Walkthrough - Analysing a Suspicious Binary and Alert Evidence" (30 Apr 2026): https://www.linkedin.com/pulse/shadow-trace-tryhackme-osman-dhaqane-9nkde

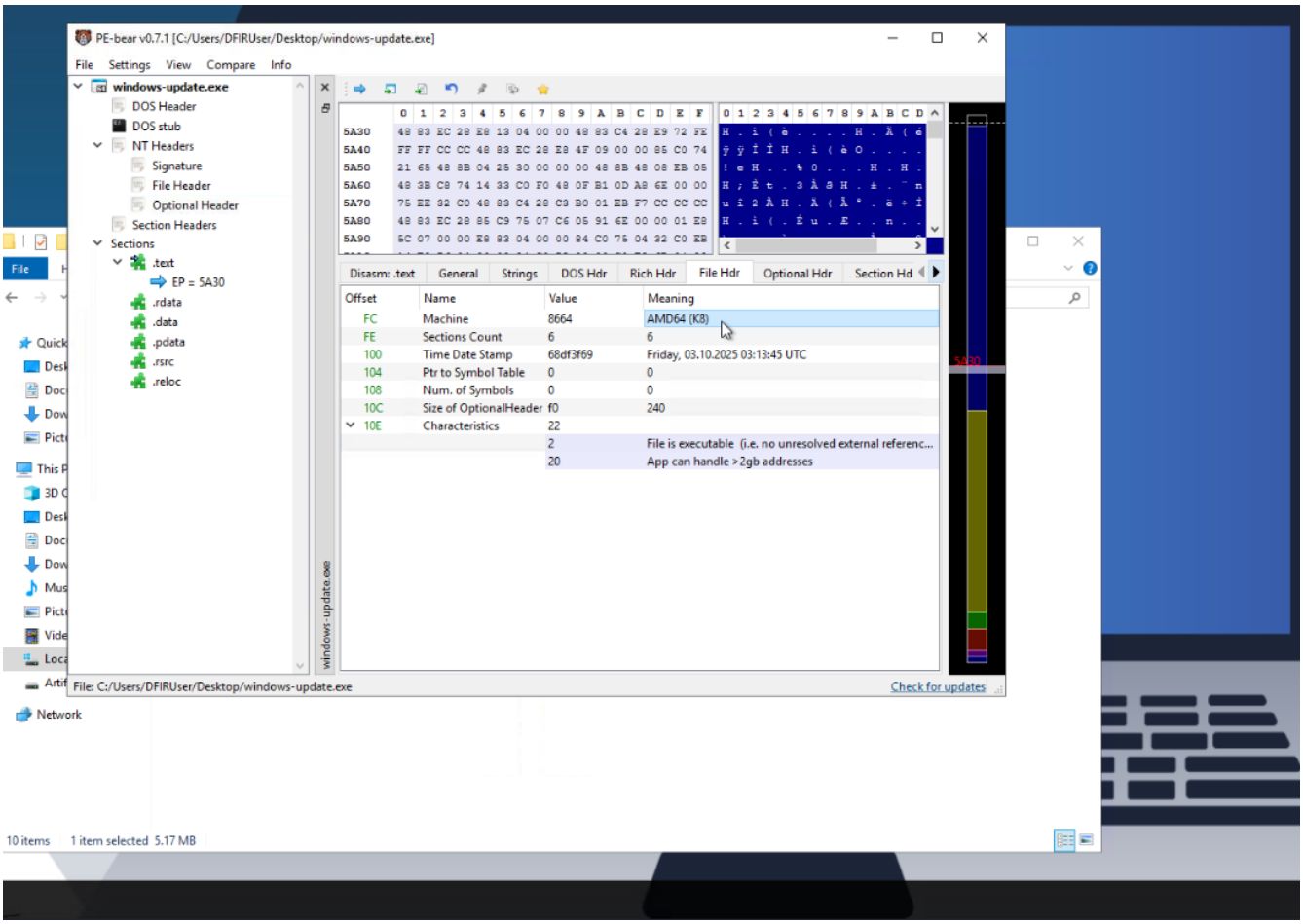
Evidence sources: user-uploaded screenshots E-01 to E-08. Public walkthrough details are clearly distinguished from what is directly visible in those screenshots.

Appendix A — Screenshot Evidence (Exact Mapping)

Each page below displays the corresponding uploaded screenshot with its unique evidence ID. These same IDs are referenced in the findings and IOC register above.

E-01 — PE-bear File Header

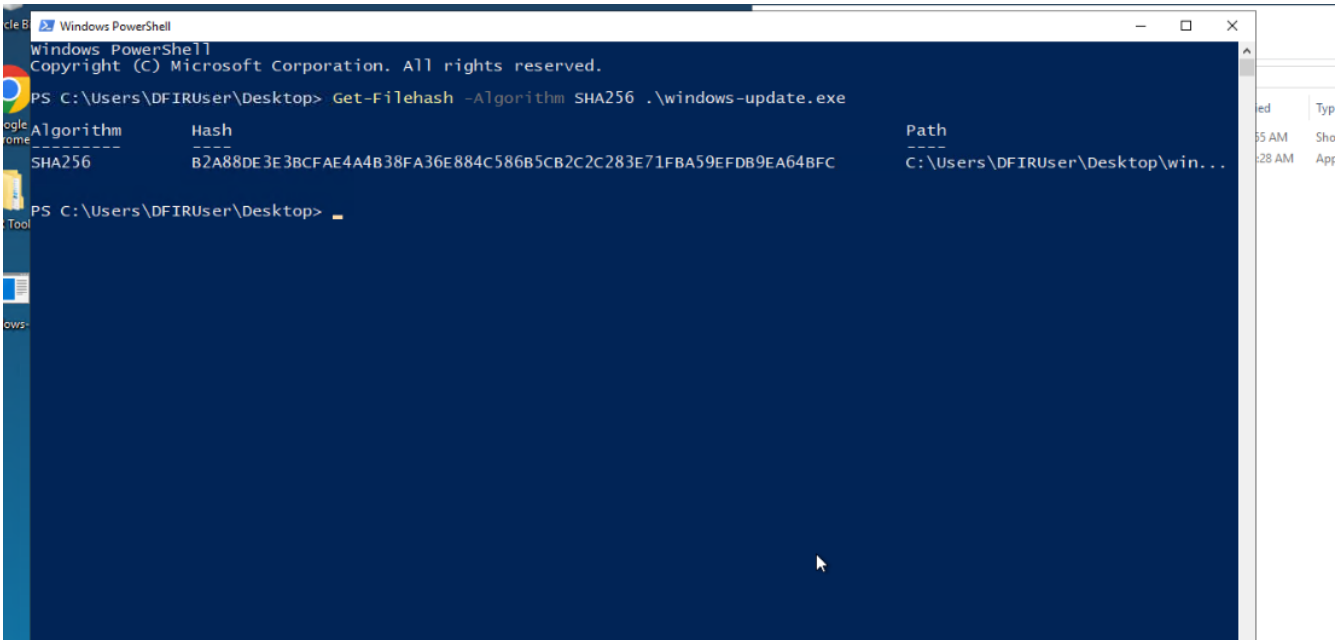
Original filename: Screenshot 2026-09-25 182222(1).png



PE-bear shows Machine value 8664 / AMD64 (x64), consistent with a 64-bit Windows PE file.

E-02 — SHA-256 hash in PowerShell

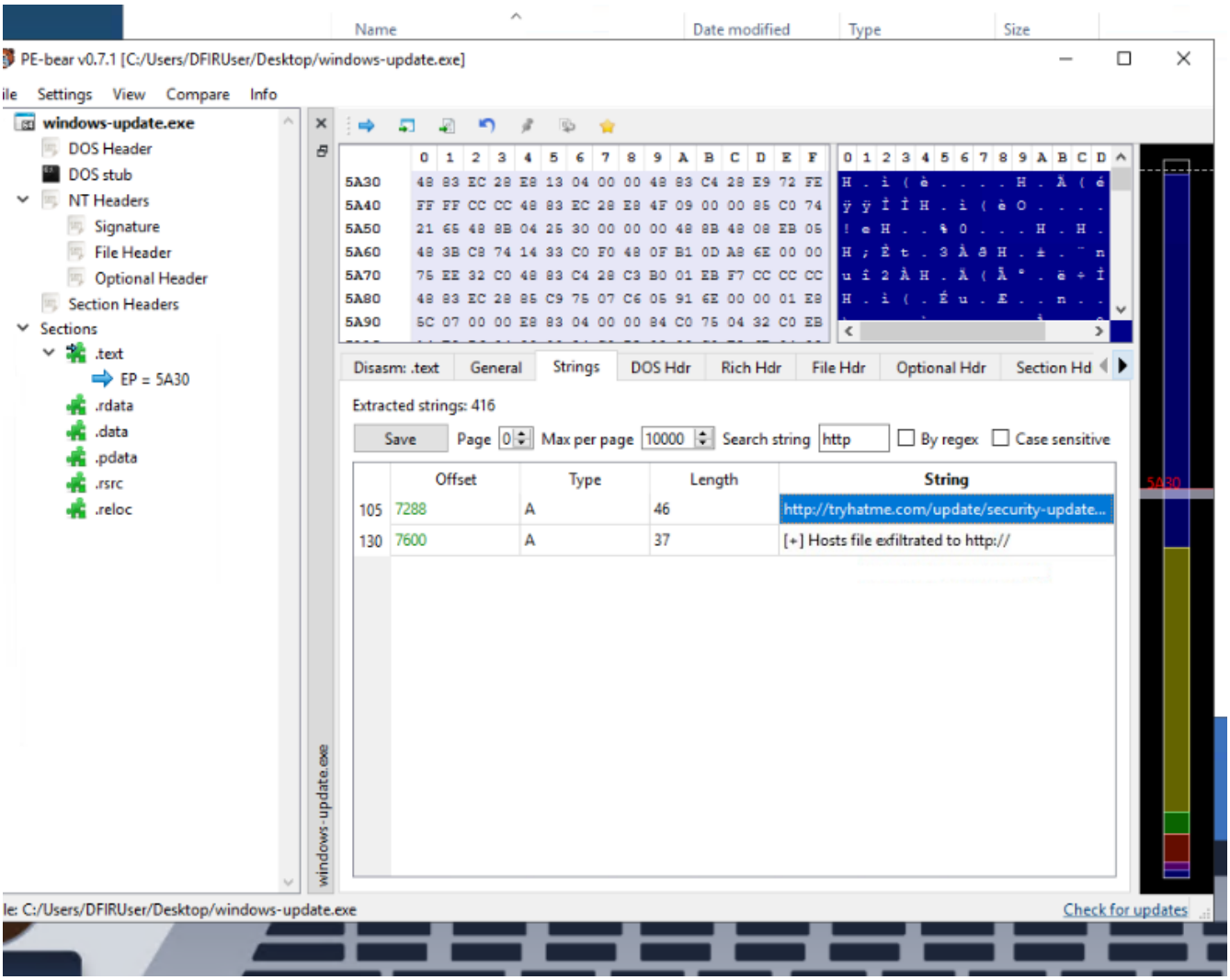
Original filename: Screenshot 2026-09-25 182535(1).png



Get-FileHash output records the SHA-256 hash for windows-update.exe.

E-03 — Embedded URL string

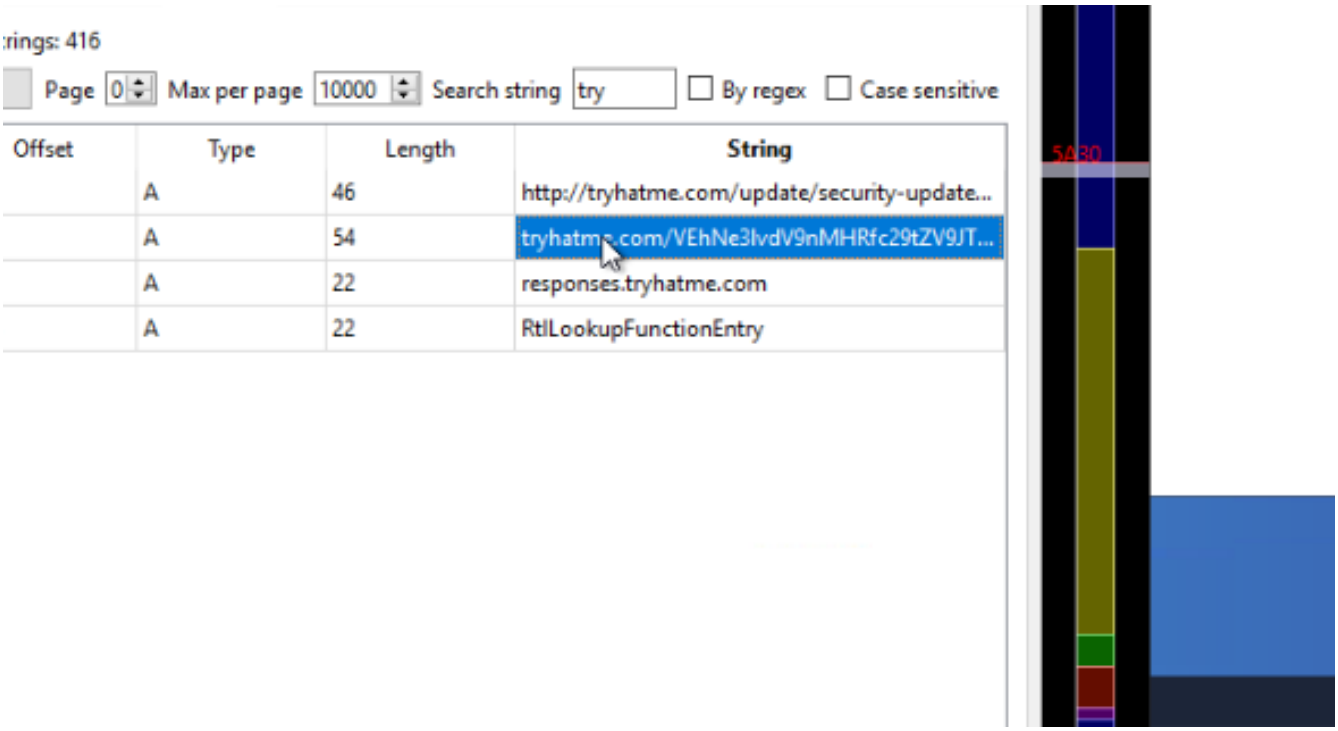
Original filename: Screenshot 2026-09-25 182643(1).png



PE-bear Strings view shows a hard-coded URL beginning http://tryhatme.com/update/security-update...

E-04 — Suspicious domain string

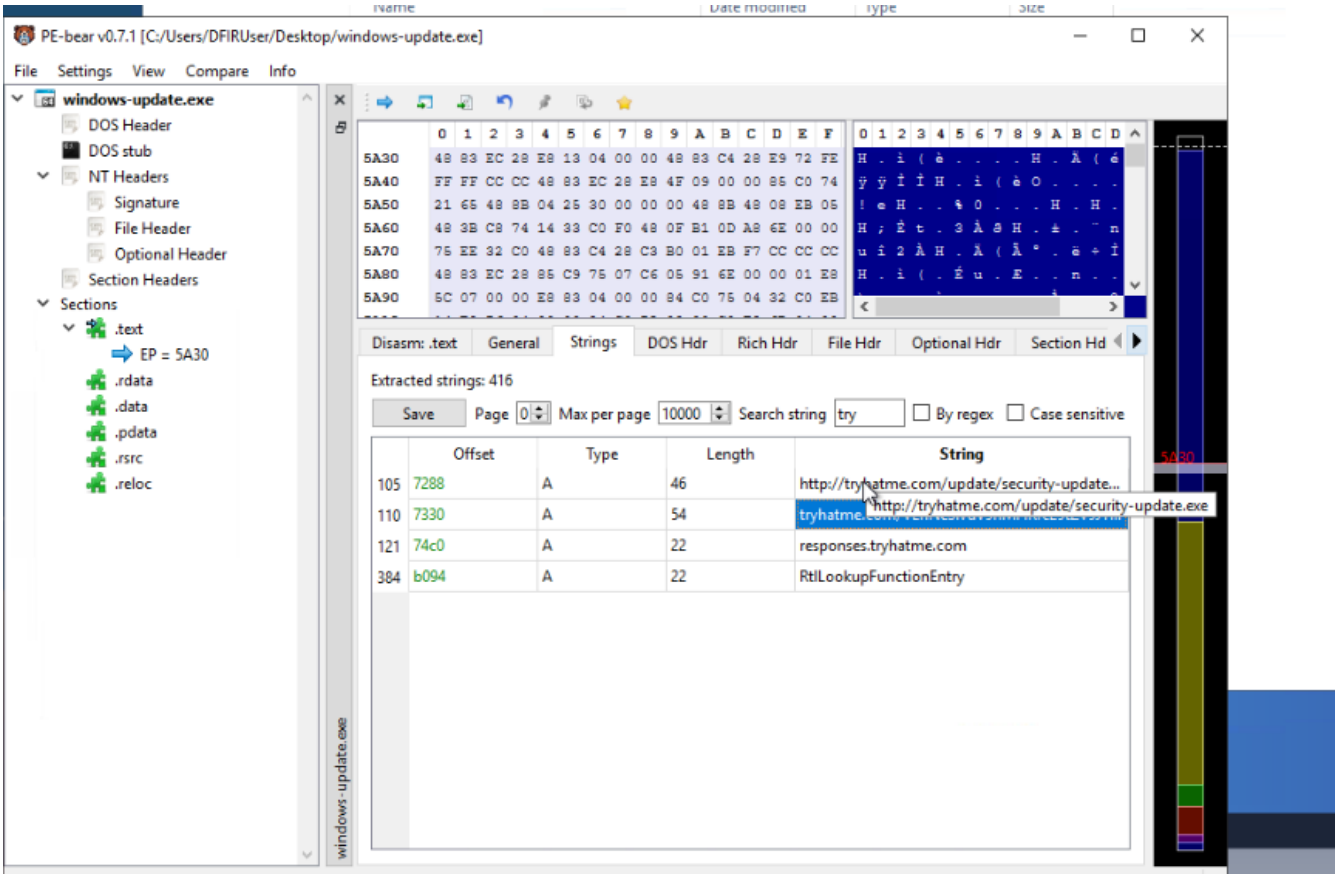
Original filename: Screenshot 2026-09-25 182804(1).png



Strings view shows an encoded/path-bearing tryhatme.com string and other strings; walkthroughs identify responses.tryhatme.com as an IOC.

E-05 — URL string in PE-bear

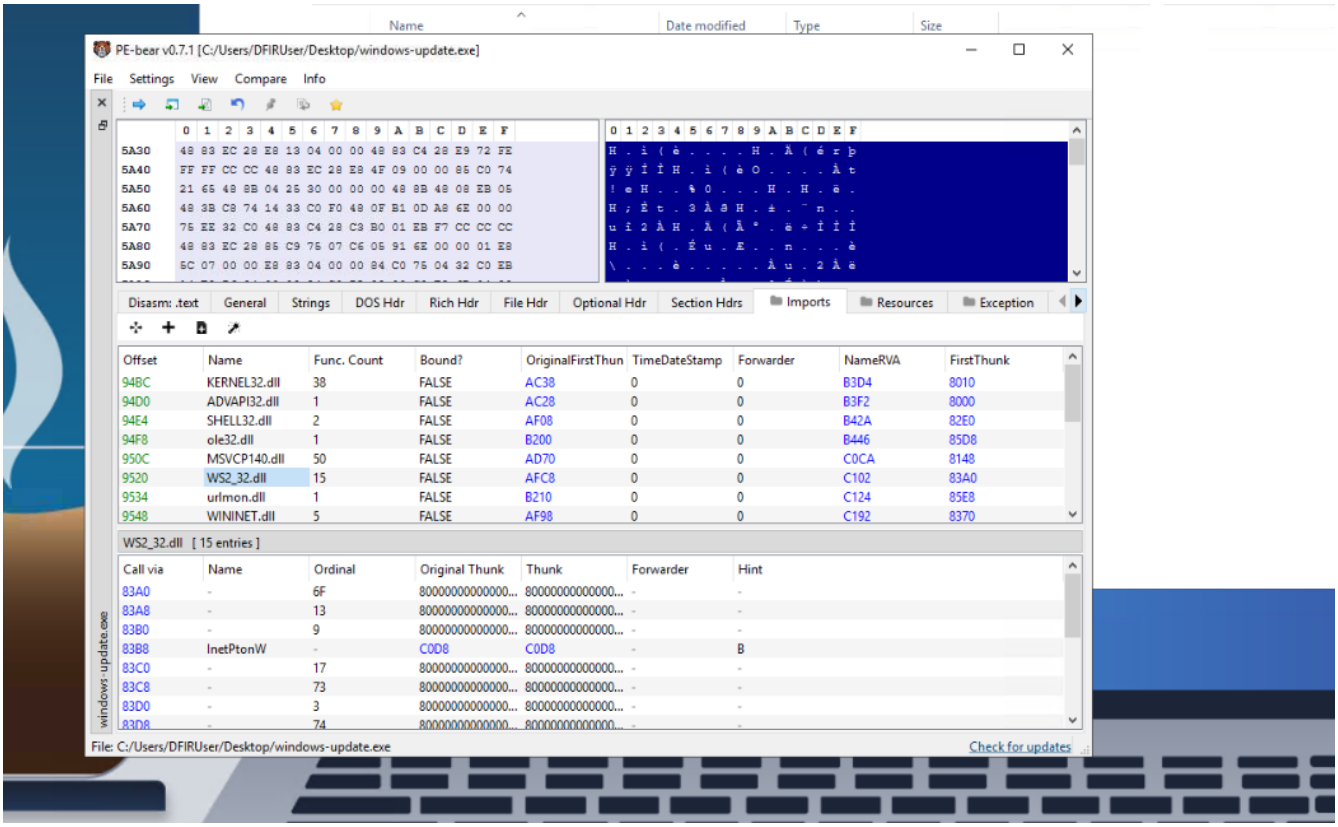
Original filename: Screenshot 2026-09-25 182811(1).png



A wider view of extracted strings shows the updater-like URL embedded in the executable.

E-06 — Imported networking library

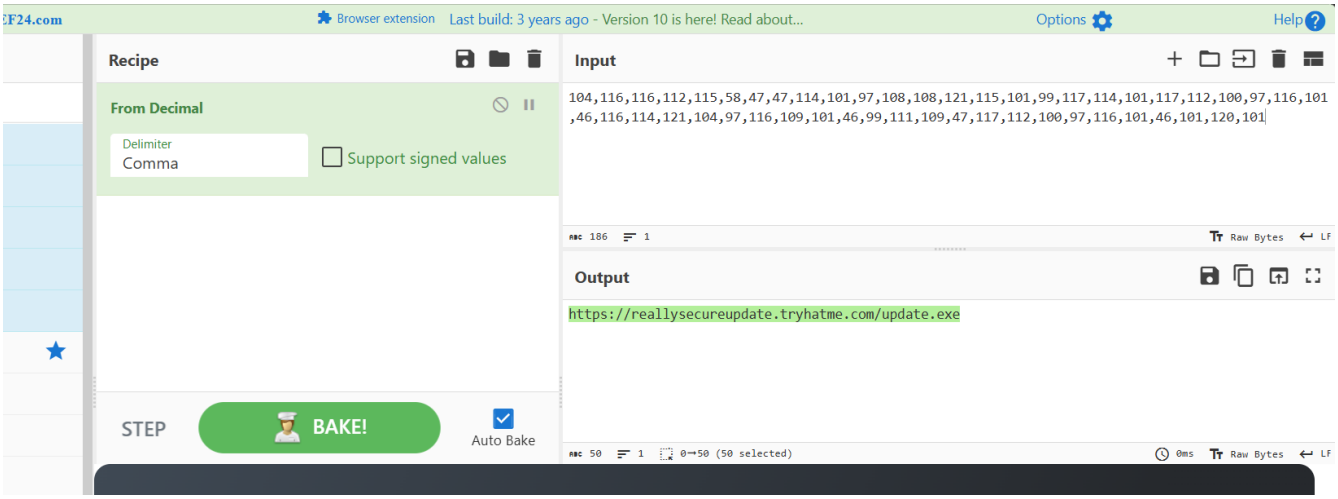
Original filename: Screenshot 2026-09-25 183310(1).png



PE-bear Imports tab lists WS2_32.dll, a Windows socket/networking library.

E-07 — CyberChef decimal decoding

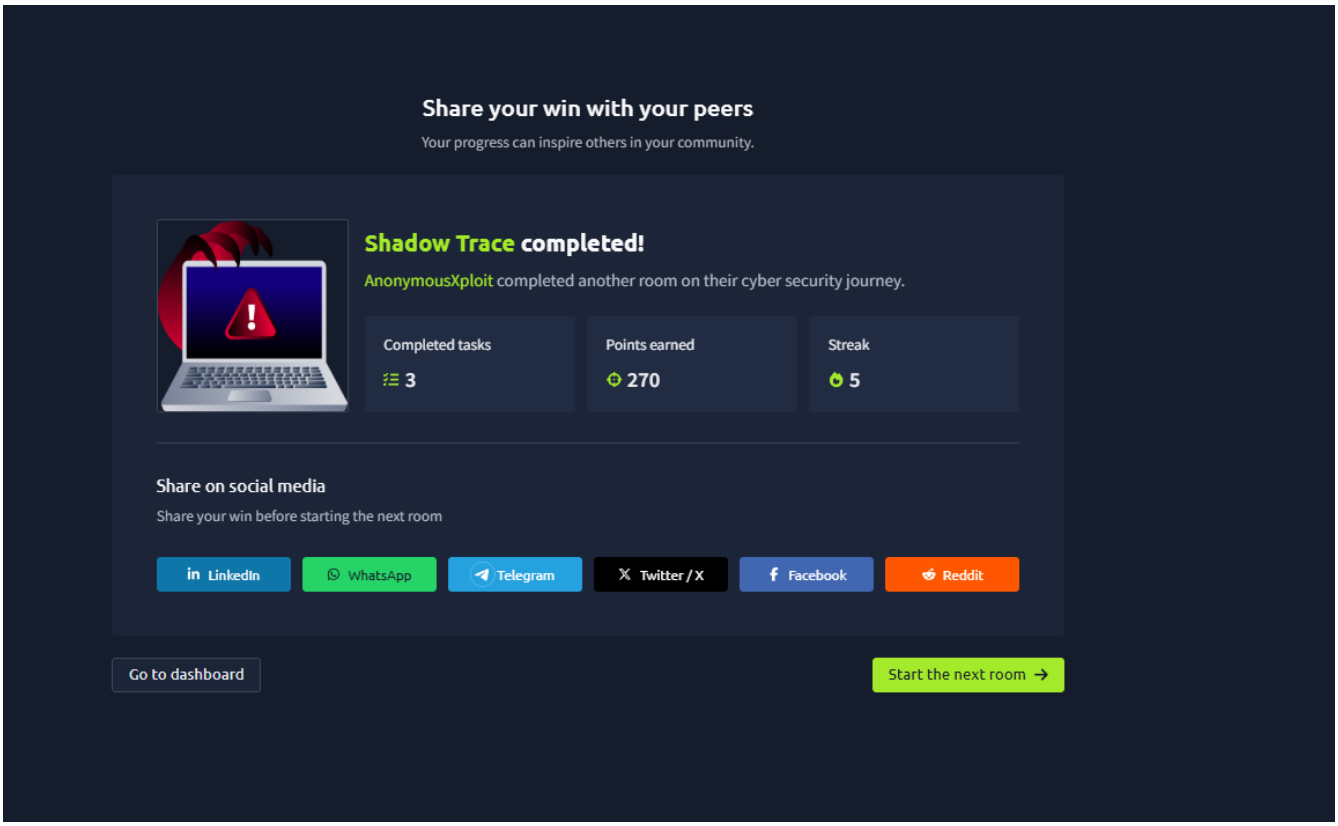
Original filename: Screenshot 2026-09-25 183816(1).png



CyberChef converts decimal character codes to https://reallysecureupdate.tryhatme.com/update.exe.

E-08 — TryHackMe completion

Original filename: Screenshot 2026-09-25 183923(1).png



The screenshot records completion of the Shadow Trace room.